

Trabalho Prático - Sistemas Operativos

Sistema de Gestão de Ficheiros

Henrique Pereira (35009) - Diogo Marques (35007) - António Torres (35002)
Rúben Andrade (34997)

Introdução

O presente trabalho tem como objetivo a aplicação prática dos conceitos de gestão de processos, ficheiros e utilização de containers no sistema operativo Linux, recorrendo a ferramentas como GCC e Docker.

Foi dada especial importância à utilização de chamadas ao sistema (system calls), evitando o uso de funções da biblioteca padrão que não eram permitidas.

Foi desenvolvido um conjunto de comandos em linguagem C utilizando chamadas ao sistema, bem como um interpretador de comandos personalizado capaz de executar esses programas, incluindo suporte a redirecionamentos e pipes.

Parte 1 – Implementação dos Comandos

Todos os comandos foram implementados utilizando exclusivamente chamadas ao sistema do Linux, nomeadamente `open()`, `read()`, `write()` e `close()`.

O tratamento de erros foi realizado através do descritor `stderr`.

Foram desenvolvidos os seguintes comandos:

- **mostra** – Apresenta o conteúdo de um ficheiro.
O comando **mostra** foi implementado utilizando as chamadas `open()`, `read()` e `write()`.
O ficheiro é aberto em modo de leitura e o seu conteúdo é lido em blocos para um buffer e posteriormente escrito para o `stdout`.

```
% mostra exemplo.txt
ola mundo
Terminou comando mostra com código 0
```

- **copia** – Cria uma cópia de um ficheiro.

O comando copia utiliza open() para abrir os ficheiros de origem e destino, e read()/write() para copiar o conteúdo entre ficheiros.

```
% copia exemplo.txt copia.txt
Terminou comando copia com código 0
% mostra copia.txt
ola mundo
Terminou comando mostra com código 0
```

- **acrescenta** – Acrescenta o conteúdo de um ficheiro a outro.
O comando acrescenta foi implementado utilizando as chamadas open(), read() e write().
O ficheiro de origem é aberto em modo de leitura e o ficheiro de destino é aberto com a flag O_APPEND, garantindo que o conteúdo é acrescentado ao final sem sobrescrever dados existentes.

O conteúdo da origem é lido em blocos e escrito sequencialmente no destino.

```
% mostra exemplo2.txt
ola mundo 2
Terminou comando mostra com código 0
% acrescenta exemplo.txt exemplo2.txt
Terminou comando acrescenta com código 0
% mostra exemplo2.txt
ola mundo 2
ola mundo
Terminou comando mostra com código 0
```

- **conta** – Conta linhas, palavras e caracteres
O comando conta percorre o conteúdo carácter a carácter, identificando linhas, palavras e caracteres através da verificação de espaços e quebras de linha.

```
% conta exemplo.txt
Linhas: 1
Palavras: 2
Caracteres: 10
Terminou comando conta com código 0
```

- **apaga** – Remove um ficheiro.

O comando apaga utiliza a chamada ao sistema unlink(), responsável por remover um ficheiro do sistema de ficheiros.

Caso o ficheiro não exista, é apresentada uma mensagem de erro ao utilizador através do descritor stderr.

```
% apaga copia.txt
Terminou comando apaga com código 0
% apaga naoexiste.txt
Erro ao apagar: No such file or directory
Terminou comando apaga com código 1
```

- informa – Mostra informação detalhada do ficheiro.

O comando informa recorre à chamada ao sistema `stat()` para obter informação detalhada sobre o ficheiro.

São apresentados dados como o tipo de ficheiro, número de i-node, tamanho, permissões, utilizador proprietário e datas de acesso, modificação e criação. A informação das permissões é convertida para formato simbólico (por exemplo `rw-r--r--`) para melhor interpretação.

```
% informa exemplo.txt
Inode: 5023
Tamanho: 10 bytes
Dono: hpereira07
Permissões: rw-r--r--
Última modificação: Fri May 15 22:21:46 2026
Terminou comando informa com código 0
```

- lista – Lista conteúdos de uma diretoria.

O comando lista foi implementado utilizando as funções `opendir()`, `readdir()` e `closedir()`.

Estas permitem percorrer todos os elementos de uma diretoria, distinguindo ficheiros de subdiretorias.

Cada entrada é lida individualmente e apresentada no ecrã, indicando o seu tipo.

```
% lista TESTE
[DIR] .
[FILE] ex3.txt
[FILE] ex1.txt
[DIR] ..
[FILE] ex2.txt
Terminou comando lista com código 0
```

- procura – Procura padrões dentro de um ficheiro ou stdin.

O comando procura permite encontrar linhas que contenham um determinado padrão. Foi implementado utilizando `read()` para ler o conteúdo carácter a carácter, construindo linhas em memória.

Cada linha é comparada com o padrão usando funções de manipulação de strings, sendo apresentada caso exista correspondência.

O comando suporta também leitura a partir do stdin, permitindo a utilização em pipes.

```
% procura exemplo.txt ola
Linha 1: ola mundo
Terminou comando procura com código 0
```

Parte 2 – Interpretador de Comandos

Foi desenvolvido um interpretador que permite a execução dos comandos implementados.

O interpretador apresenta o símbolo '%' como prompt e lê comandos introduzidos pelo utilizador, funciona de forma semelhante a um shell simplificado do Linux.

A execução dos comandos é feita através da função `fork()`, que cria um novo processo filho.

No processo filho é utilizada a função `execvp()` para executar o comando.

O processo pai utiliza `waitpid()` para aguardar a conclusão do comando e obter o seu código de saída.

Funcionalidades implementadas:

- Execução de comandos
- Redirecionamento de entrada (<)
- Redirecionamento de saída (>)
- Pipes entre dois comandos

```
% lista | conta
Linhas: 26
Palavras: 52
Caracteres: 397
Terminou pipe com código 0
% lista > saida.txt
Terminou comando lista com código 0
% mostra saida.txt
[DIR] .
[FILE] procura.c
[FILE] procura
[FILE] apaga
[FILE] mostra.c
[FILE] INTERPRETADOR
```

Os pipes foram implementados recorrendo à chamada `pipe()`, permitindo redirecionar a saída de um comando para a entrada de outro.

Foram utilizados descritores de ficheiros e a função `dup2()` para redirecionar `stdin` e `stdout`.

No interpretador também foi implementado um comando “termina” que após ser utilizado termina o interpretador e retorna à shell do sistema operativo.

Este comando é identificado através da comparação da string introduzida pelo utilizador com a palavra “termina”, utilizando a função `strcmp()`.

```
% termina
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ |
```

Parte 3 – Docker

Foi criado um Dockerfile baseado na imagem Ubuntu que permite compilar e executar o interpretador dentro de um container.

Durante o processo de construção da imagem, o código fonte é copiado para o container e compilado automaticamente através do compilador `gcc`.

O interpretador foi definido como `ENTRYPOINT`, garantindo que o container inicia diretamente no interpretador.

```
# imagem base
FROM ubuntu:22.04

# evitar prompts
ENV DEBIAN_FRONTEND=noninteractive

# instalar gcc
RUN apt-get update && apt-get install -y gcc make

# criar pasta de trabalho
WORKDIR /app

# copiar tudo
COPY . .

# compilar todos os programas
RUN gcc mostra.c -o mostra && \
    gcc copia.c -o copia && \
    gcc acrescenta.c -o acrescenta && \
    gcc conta.c -o conta && \
    gcc apaga.c -o apaga && \
    gcc informa.c -o informa && \
    gcc lista.c -o lista && \
    gcc procura.c -o procura && \
    gcc INTERPRETADOR.c -o INTERPRETADOR

# adicionar ao PATH
ENV PATH="/app:${PATH}"

# comando de entrada
ENTRYPOINT ["/INTERPRETADOR"]
```

Construção da imagem e Execução do Container:

`docker build -t interpretador-img .`

`docker run -it interpretador-img`

```
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ sudo docker build -t interpretador-img .
[+] Building 30.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 684B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/ubuntu:22.04@sha256:13861e4d4047fbbe1fc1737d690dffe8d31c4524c8f26
=> => resolve docker.io/library/ubuntu:22.04@sha256:13861e4d4047fbbe1fc1737d690dffe8d31c4524c8f26
=> => sha256:40d16f30db405106ef8074779bdf41f012465c2a785bbeaa2eab9f2081099b47 29.74MB / 29.74MB
=> => extracting sha256:40d16f30db405106ef8074779bdf41f012465c2a785bbeaa2eab9f2081099b47
=> [internal] load build context
=> => transferring context: 157.86kB
=> [2/5] RUN apt-get update && apt-get install -y gcc make
=> [3/5] WORKDIR /app
=> [4/5] COPY . .
=> [5/5] RUN gcc mostra.c -o mostra && gcc copia.c -o copia && gcc acrescenta.c -o acresc
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:76e038518f64c2c87a8fb8028f057d57b54d6c8348ada8ab6814c979386fbcad
=> => exporting config sha256:086b29ddd79603ee293620e58c440b941a69512a06fcdd448921f42cbce66158
=> => exporting attestation manifest sha256:616b7c50615de7793a740eb737cbe6e6571141b87d68969455c1e
=> => exporting manifest list sha256:7256544f97e4b8a9cc3af8924efcc609d3c99dbbe2bb5d2c4113363badb2
=> => naming to docker.io/library/interpretador-img:latest
=> => unpacking to docker.io/library/interpretador-img:latest
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ sudo docker run -it interpretador-img
%
```

Parte 4 – Análise ao Sistema de Ficheiros

O ficheiro `fs.img` foi montado utilizando '`mount -o loop`'. A análise permitiu identificar o tipo de sistema de ficheiros e o conteúdo presente.

Para obter a informação detalhada do sistema de ficheiros foi utilizado o comando `stat -f`.

A) Tipo de sistema de ficheiros: `msdos`

Tamanho do bloco: `2048 bytes`

Número de inodes: `0`

```
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos/fs$ stat -f .
File: "."
ID: 700000000000 Namelen: 1530 Type: msdos
Block size: 2048 Fundamental block size: 2048
Blocks: Total: 502 Free: 501 Available: 501
Inodes: Total: 0 Free: 0
```

B) Conteúdo do ficheiro disponível: `ficheiro1`

```
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ mkdir fs
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ sudo mount -o loop fs.img fs
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos$ cd fs
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos/fs$ ls
ficheiro1
```

O conteúdo do ficheiro foi obtido através do comando `cat ficheiro1`, verificando-se o seguinte:

```
hpereira07@DESKTOP-90GRMH4:~/Projeto/Comandos/fs$ cat ficheiro1  
ficheiro1
```

Apesar de o enunciado referir o ficheiro ipca.txt, este não se encontra presente na imagem fornecida. Em alternativa, foi analisado o ficheiro disponível (ficheiro1), conforme apresentado.

Conclusão

Este trabalho permitiu consolidar conhecimentos sobre chamadas ao sistema, gestão de processos e utilização de containers. A implementação prática demonstrou a interação entre processos e a importância do controlo de entradas e saídas.

Uma das principais dificuldades foi a implementação correta dos pipes, nomeadamente a gestão dos descritores de ficheiros. Após umas pequenas mudanças de código foi possível implementar uma solução funcional. Este trabalho permitiu também compreender o funcionamento interno de um sistema operativo ao nível da gestão de ficheiros e processos.